

Assignment 3 extra

Hadeer Ghoneim

Verification Diploma

Q1

```

1  module queue_test;
2
3      // Declare integer variable and queue
4      int j;
5      int q[$]; // queue of integers
6
7      initial begin
8          // Step 1: Initialize j = 1 and queue q = {0, 2, 5}
9          j = 1;
10         q = {0, 2, 5};
11         $display("Initial Queue q = %p", q);
12
13         // Step 2: Insert j at index 1
14         q.insert(1, j);
15         $display("After inserting j=%0d at index 1: q = %p", j, q);
16
17         // Step 3: Delete element at index 1
18         q.delete(1);
19         $display("After deleting index 1: q = %p", q);
20
21         // Step 4: Push element 7 at the front
22         q.push_front(7);
23         $display("After pushing 7 at the front: q = %p", q);
24
25         // Step 5: Push element 9 at the back
26         q.push_back(9);
27         $display("After pushing 9 at the back: q = %p", q);
28
29         // Step 6: Pop element from the back into j
30         j = q.pop_back();
31         $display("After popping from back: q = %p, j = %0d", q, j);
32
33         // Step 7: Pop element from the front into j
34         j = q.pop_front();
35         $display("After popping from front: q = %p, j = %0d", q, j);
36

```

```

36
37 // Step 8: Reverse the queue
38 q.reverse();
39 $display("After reverse: q = %p", q);
40
41 // Step 9: Sort the queue
42 q.sort();
43 $display("After sort: q = %p", q);
44
45 // Step 10: Reverse sort (rsort) the queue
46 q.rsort();
47 $display("After reverse sort (rsort): q = %p", q);
48
49 // Step 11: Shuffle the queue
50 q.shuffle();
51 $display("After shuffle: q = %p", q);
52
53 // Finish simulation
54 $finish;
55 end
56
57 endmodule
58

```

Transcript

File Edit View Bookmarks Window Help

Transcript

```

# Reading pref.tcl
# // Questa Sim-64
# // Version 2021.1 win64 Jan 19 2021
# //
# // Copyright 1991-2021 Mentor Graphics Corporation
# // All Rights Reserved.
# //
# // QuestaSim and its associated documentation contain trade
# // secrets and commercial or financial information that are the property of
# // Mentor Graphics Corporation and are privileged, confidential,
# // and exempt from disclosure under the Freedom of Information Act,
# // 5 U.S.C. Section 552. Furthermore, this information
# // is prohibited from disclosure under the Trade Secrets Act,
# // 18 U.S.C. Section 1905.
# //
# Loading project aas_3_q_4
# reading C:/Users/General/Documents/questasim64_2021.1/win64/./modelsim.ini
# Loading project extra_3_q3
# Compile of queue_test.sv was successful.
QuestaSim> vsim -voptargs="+acc" work.queue_test
# vsim -voptargs="+acc" work.queue_test
# Start time: 02:30:49 on Sep 12, 2025
# ** Note: (vsim-3812) Design is being optimized...
# Loading sv_std.std
# Loading work.queue_test(fast)
VSIM2> run -all
# Initial Queue q = '{0, 2, 5}
# After inserting j=1 at index 1: q = '{0, 1, 2, 5}
# After deleting index 1: q = '{0, 2, 5}
# After pushing 7 at the front: q = '{7, 0, 2, 5}
# After pushing 9 at the back: q = '{7, 0, 2, 5, 9}
# After popping from back: q = '{7, 0, 2, 5}, j = 9
# After popping from front: q = '{0, 2, 5}, j = 7
# After reverse: q = '{5, 2, 0}
# After sort: q = '{0, 2, 5}
# After reverse sort (rsort): q = '{5, 2, 0}
# After shuffle: q = '{2, 0, 5}
# ** Note: $finish : C:/Users/General/Desktop/.github/build/verification diploma/verification assignments/assignment 3 extra/Q1/queue_test.sv(59)
# Time: 0 ns Iteration: 0 Instance: /queue_test
# 1
# Break in Module queue_test at C:/Users/General/Desktop/.github/build/verification diploma/verification assignments/assignment 3 extra/Q1/queue_test.sv line 59
VSIM3>

```

Q2

1. Testbench code

```
1  module adder_tb;
2
3      // Import the package
4      import adder_pkg::*;
5
6      // Clock and reset
7      bit clk;
8      bit reset;
9
10     // DUT signals
11     logic signed [3:0] A_tb, B_tb;
12     logic signed [4:0] C_tb;
13
14     // Counters
15     int error_count;
16     int correct_count;
17
18     // DUT instantiation
19     adder dut (
20         .clk(clk),
21         .reset(reset),
22         .A(A_tb),
23         .B(B_tb),
24         .C(C_tb)
25     );
26
27     // Clock generation
28     initial clk = 0;
29     always #5 clk = ~clk;
30
31     // Create object from adder_transaction
32     adder_transaction t;
33
```

```

34 // Initial block
35 initial begin
36     error_count    = 0;
37     correct_count = 0;
38
39     // Initialize transaction object
40     t = new();
41
42     // Apply initial reset
43     reset = 1;
44     A_tb  = 0;
45     B_tb  = 0;
46     @(posedge clk);
47     reset = 0;
48
49     // Run random tests
50     repeat (10000) begin
51         // Randomize the transaction
52         assert(t.randomize()) else $fatal("Randomization failed!");
53
54         // Drive DUT inputs
55         @(negedge clk);
56         reset = t.reset; // Random reset with low probability
57         A_tb  = t.A;
58         B_tb  = t.B;
59
60         // Sample coverage only when reset is de-asserted
61         @(posedge clk);
62         t.sample();
63

```

```

63
64     // Self-checking: verify the result
65     if (!reset) begin
66         if (C_tb !== (A_tb + B_tb)) begin
67             $display("ERROR: A=%0d, B=%0d, Expected=%0d, Got=%0d", A_tb, B_tb, (A_tb+B_tb), C_tb);
68             error_count++;
69         end else begin
70             correct_count++;
71         end
72     end
73 end
74
75 // Final report
76 $display("=====");
77 $display("Simulation Finished!");
78 $display("Correct operations: %0d", correct_count);
79 $display("Errors detected:   %0d", error_count);
80 $display("=====");
81
82 $stop;
83 end
84
85 endmodule
86

```

2. Package code

```
1 package adder_pkg;
2
3 // Enum
4 typedef enum logic signed [3:0] {
5     MAXNEG = -4'sd8,
6     ZERO   = 4'sd0,
7     MAXPOS = 4'sd7
8 } special_values_e;
9
10 // Class
11 class adder_transaction;
12
13     // Randomized inputs
14     rand logic signed [3:0] A;
15     rand logic signed [3:0] B;
16     rand bit reset;
17
18     // Reset happens rarely
19     constraint c_reset_low_prob {
20         reset dist {1 := 1, 0 := 9};
21     }
22
23     // A more likely to be MAXPOS, ZERO, MAXNEG
24     constraint c_A_values {
25         A dist {-4'sd8 := 5, 4'sd0 := 5, 4'sd7 := 5, [-8:7] := 1};
26     }
27
28     // B more likely to be MAXPOS, ZERO, MAXNEG
29     constraint c_B_values {
30         B dist {-4'sd8 := 5, 4'sd0 := 5, 4'sd7 := 5, [-8:7] := 1};
31     }
32
```

```

32
33 // Functional coverage for A
34 covergroup Covgrp_A;
35     cp_A_values: coverpoint A {
36         bins data_0      = {ZERO};
37         bins data_max    = {MAXPOS};
38         bins data_min    = {MAXNEG};
39         bins data_default = default;
40     }
41     cp_A_transitions: coverpoint A {
42         bins data_0max    = (ZERO => MAXPOS);
43         bins data_maxmin  = (MAXPOS => MAXNEG);
44         bins data_minmax  = (MAXNEG => MAXPOS);
45     }
46 endgroup
47
48 // Functional coverage for B
49 covergroup Covgrp_B;
50     cp_B_values: coverpoint B {
51         bins data_0      = {ZERO};
52         bins data_max    = {MAXPOS};
53         bins data_min    = {MAXNEG};
54         bins data_default = default;
55     }
56     cp_B_transitions: coverpoint B {
57         bins data_0max    = (ZERO => MAXPOS);
58         bins data_maxmin  = (MAXPOS => MAXNEG);
59         bins data_minmax  = (MAXNEG => MAXPOS);
60     }
61 endgroup
62

```

```

62
63     // Constructor
64     function new();
65         Covgrp_A = new();
66         Covgrp_B = new();
67     endfunction
68
69     // Manual sampling
70     function void sample();
71         if (!reset) begin
72             Covgrp_A.sample();
73             Covgrp_B.sample();
74         end
75     endfunction
76
77 endclass : adder_transaction
78
79 endpackage : adder_pkg
80

```

3. Design code

```

1  module adder (
2      input  clk,
3      input  reset,
4      input  signed [3:0] A, // Input data A in 2's complement
5      input  signed [3:0] B, // Input data B in 2's complement
6      output reg signed [4:0] C // Adder output in 2's complement
7      );
8
9      // Register output C
10     always @(posedge clk or posedge reset) begin
11         if (reset)
12             C <= 5'b0;
13         else
14             C <= A + B;
15     end
16
17 endmodule
18

```

4. Report any bugs detected in the design and fix them

5. Snippet to your verification plan document

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ADDER_1	When reset is asserted, output C should go to 0.	Directed stimulus at the start of the simulation.	Not sampled during reset.	Checker in the testbench to ensure $C == 0$ when reset is high.
ADDER_2	Inputs A and B must include MAXPOS, ZERO, MAXNEG more frequently than other values.	Randomization using weighted constraints in class.	Covergroups Covgrp_A and Covgrp_B with bins for MAXPOS, ZERO, MAXNEG, and default values.	Verify by comparing DUT output C with golden expected output $A + B$.
ADDER_3	Verify transitions between A values: ZERO $\rightarrow$ MAXPOS, MAXPOS $\rightarrow$ MAXNEG, MAXNEG $\rightarrow$ MAXPOS.	Randomization across multiple clock cycles.	Coverpoint cp_A_transitions for required transitions.	Monitor and confirm expected transitions captured by coverage bins.
ADDER_4	Verify transitions between B values: ZERO $\rightarrow$ MAXPOS, MAXPOS $\rightarrow$ MAXNEG, MAXNEG $\rightarrow$ MAXPOS.	Randomization across multiple clock cycles.	Coverpoint cp_B_transitions for required transitions.	Monitor and confirm expected transitions captured by coverage bins.
ADDER_5	Verify addition results for all edge cases (overflow and underflow included).	Randomization + directed edge cases for MAXPOS and MAXNEG.	Use coverage to ensure all edge combinations of A and B are tested.	Self-checker compares DUT output with reference model $(A + B)$.
ADDER_6	Ensure full code coverage is achieved (e.g., reset branches, all always blocks, etc.).	Combination of directed + random stimulus.	Code coverage enabled in simulator (vsim -coverage).	Review coverage report and add extra tests if needed to reach 100%.

6. Do file

```

1  vlib work
2  vlog adder.v adder_pkg.svh adder_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.adder_tb -cover
4  add wave *
5  coverage save adder_pkg.ucdb -onexit
6  run -all

```

7. Code Coverage report snippets

cvr_adder - Notepad

File Edit Format View Help

Coverage Report by instance with details

```
=====
=== Instance: /adder_tb/dut
=== Design Unit: work.adder
=====
```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	2	2	0	100.00%

=====Branch Details=====

Branch Coverage for instance /adder_tb/dut

Line	Item	Count	Source
----	----	-----	-----
File adder.v			
-----IF Branch-----			
11		10887	Count coming in to IF
11	1	1908	if (reset)
13	1	8979	else

Branch totals: 2 hits of 2 branches = 100.00%

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	3	3	0	100.00%

=====Statement Details=====

Statement Coverage for instance /adder_tb/dut --

Line	Item	Count	Source
----	----	-----	-----
File adder.v			
1			module adder (
2			input clk,

Statement Coverage for instance /adder_tb/dut --

Line	Item	Count	Source
----	----	-----	-----
File adder.v			
1			module adder (
2			input clk,
3			input reset,
4			input signed [3:0] A, // Input data A in 2's complement
5			input signed [3:0] B, // Input data B in 2's complement
6			output reg signed [4:0] C // Adder output in 2's complement
7			);
8			
9			// Register output C
10	1	10887	always @(posedge clk or posedge reset) begin
11			if (reset)
12	1	1908	C <= 5'b0;
13			else
14	1	8979	C <= A + B;

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	30	30	0	100.00%

=====Toggle Details=====

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	30	30	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /adder_tb/dut --

Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----
A[0-3]	1	1	100.00
B[0-3]	1	1	100.00
C[4-0]	1	1	100.00
clk	1	1	100.00
reset	1	1	100.00

Total Node Count = 15
Toggled Node Count = 15
Untoggled Node Count = 0

Toggle Coverage = 100.00% (30 of 30 bins)

```

=====
=== Instance: /adder_tb
=== Design Unit: work.adder_tb
=====

```

```

Assertion Coverage:
  Assertions              1          1          0  100.00%

```

```

-----
Name                File(Line)                Failure Pass
Count              Count
-----
/adder_tb/#ublk#179077330#50/immed_52
              adder_tb.svh(52)                0          1

```

```

Branch Coverage:
  Enabled Coverage      Bins    Hits    Misses Coverage
-----
  Branches              4       4       0    100.00%

```

```

=====Branch Details=====

```

```

Branch Coverage for instance /adder_tb

```

```

  Line      Item              Count      Source
  ----      -
File adder_tb.svh
-----IF Branch-----
  65              10000    Count coming in to IF
  65              8992      if (!reset) begin
                                1008    All False Count
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
  66              8992    Count coming in to IF
  66              8497      if (C_tb !== (A_tb + B_tb)) begin
  69              495      end else begin
Branch totals: 2 hits of 2 branches = 100.00%

```

Condition Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
-----	----	----	-----	-----
Conditions	1	1	0	100.00%

=====Condition Details=====

Condition Coverage for instance /adder_tb --

File adder_tb.svh

-----Focused Condition View-----

Line 66 Item 1 (C_tb !== (A_tb + B_tb))

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
-----	-----	-----	-----
(C_tb !== (A_tb + B_tb))	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-----	-----	-----	-----
Row 1:	1	(C_tb !== (A_tb + B_tb))_0	-
Row 2:	1	(C_tb !== (A_tb + B_tb))_1	-

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	27	27	0	100.00%

=====Statement Details=====

8. Functional Coverage report snippets

Covergroups

Covergroups										
File Bookmarks Window Help										
Covergroups										
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment	
/adder_pkg/adder...		100.00%								
TYPE Covgrp...		100.00%	100	100.00...		✓	auto(0)			
CVP Covgr...		100.00%	100	100.00...		✓				
CVP Covgr...		100.00%	100	100.00...		✓				
INST V\ad...		100.00%	100	100.00...		✓			0	
CVP cp...		100.00%	100	100.00...		✓				
bin ...		2060	1	100.00...		✓				
bin ...		2072	1	100.00...		✓				
bin ...		343	1	100.00...		✓				
def...		4517	-	-		✓				
CVP cp...		100.00%	100	100.00...		✓				
bin ...		456	1	100.00...		✓				
bin ...		74	1	100.00...		✓				
bin ...		77	1	100.00...		✓				
TYPE Covgrp...		100.00%	100	100.00...		✓	auto(0)			
CVP Covgr...		100.00%	100	100.00...		✓				
CVP Covgr...		100.00%	100	100.00...		✓				
INST V\ad...		100.00%	100	100.00...		✓			0	
CVP cp...		100.00%	100	100.00...		✓				
bin ...		2088	1	100.00...		✓				
bin ...		2163	1	100.00...		✓				
bin ...		363	1	100.00...		✓				
def...		4378	-	-		✓				
CVP cp...		100.00%	100	100.00...		✓				
bin ...		496	1	100.00...		✓				
bin ...		85	1	100.00...		✓				
bin ...		94	1	100.00...		✓				

cvr_adder - Notepad

File Edit Format View Help

Covergroup	Metric	Goal	Bins	Status
TYPE /adder_pkg/adder_transaction/Covgrp_A	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Coverpoint cp_A_values	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Coverpoint cp_A_transitions	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Covergroup instance \adder_pkg::adder_transaction::Covgrp_A	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Coverpoint cp_A_values	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin data_0	2060	1	-	Covered
bin data_max	2072	1	-	Covered
bin data_min	343	1	-	Covered
default bin data_default	4517		-	Occurred
Coverpoint cp_A_transitions	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin data_0max	456	1	-	Covered
bin data_maxmin	74	1	-	Covered
bin data_minmax	77	1	-	Covered
TYPE /adder_pkg/adder_transaction/Covgrp_B	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Coverpoint cp_B_values	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	

cvr_adder - Notepad

File Edit Format View Help

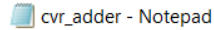
% Hit:	100.00%	100	-	
Coverpoint cp_B_transitions	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Covergroup instance \/_adder_pkg::adder_transaction::Covgrp_B				
	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Coverpoint cp_B_values	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin data_0	2088	1	-	Covered
bin data_max	2163	1	-	Covered
bin data_min	363	1	-	Covered
default bin data_default	4378		-	Occurred
Coverpoint cp_B_transitions	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin data_0max	496	1	-	Covered
bin data_maxmin	85	1	-	Covered
bin data_minmax	94	1	-	Covered

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	4	4	0	100.00%

COVERGROUP COVERAGE:

Covergroup	Metric	Goal	Bins	Status
TYPE /adder_pkg/adder_transaction/Covgrp_A	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Coverpoint cp_A_values	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Coverpoint cp_A_transitions	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Covergroup instance \adder_pkg::adder_transaction::Covgrp_A	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Coverpoint cp_A_values	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin data_0	2060	1	-	Covered
bin data_max	2072	1	-	Covered
bin data_min	343	1	-	Covered
default bin data_default	4517		-	Occurred
Coverpoint cp_A_transitions	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin data_0max	456	1	-	Covered
bin data_maxmin	74	1	-	Covered
bin data_minmax	77	1	-	Covered
TYPE /adder_pkg/adder_transaction/Covgrp_B	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	

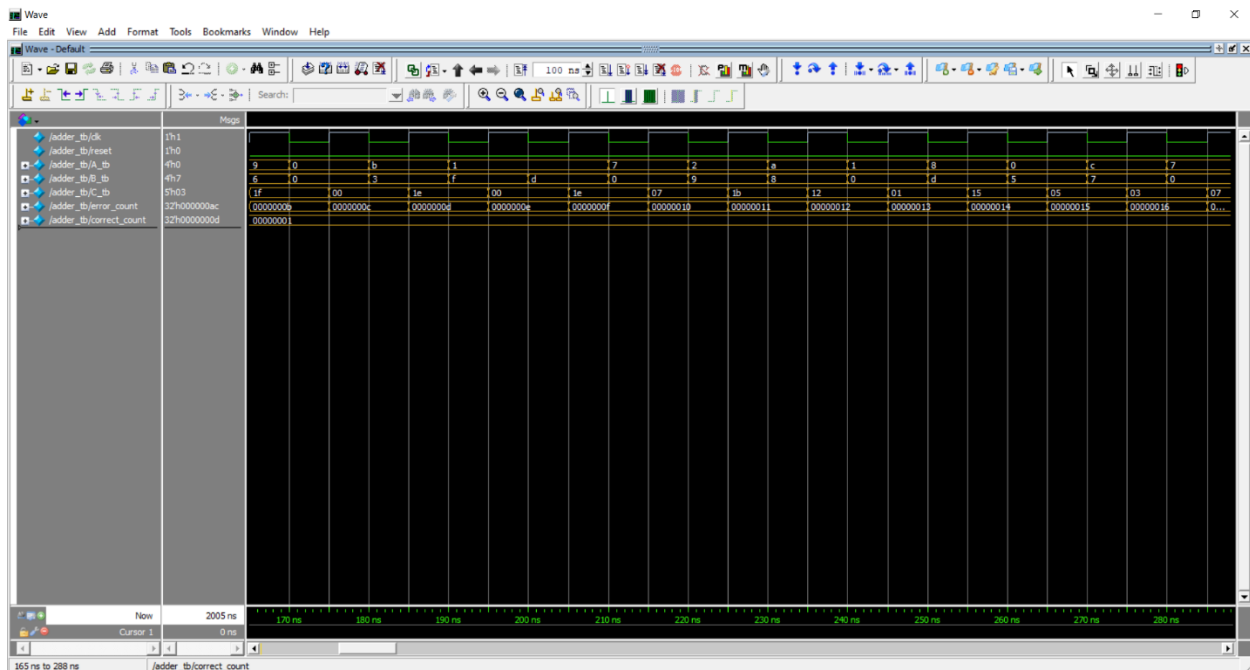


File Edit Format View Help

% Hit:	100.00%	100	-	
Coverpoint cp_B_values	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Coverpoint cp_B_transitions	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Covergroup instance \adder_pkg::adder_transaction::Covgrp_B	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Coverpoint cp_B_values	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin data_0	2088	1	-	Covered
bin data_max	2163	1	-	Covered
bin data_min	363	1	-	Covered
default bin data_default	4378		-	Occurred
Coverpoint cp_B_transitions	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin data_0max	496	1	-	Covered
bin data_maxmin	85	1	-	Covered
bin data_minmax	94	1	-	Covered

TOTAL COVERGROUP COVERAGE: 100.00% COVERGROUP TYPES: 2

9. Clear and neat QuestaSim waveform snippets showing the functionality of the design



Q3

1. Testbench code

```
1  module FSM_tb;
2
3      import fsm_pkg::*;
4
5      // DUT signals
6      logic clk, rst, x, y;
7      logic [9:0] users_count;
8
9      // Instantiate DUT
10     FSM_010 dut (.clk(clk), .rst(rst), .x(x), .y(y), .users_count(users_count));
11
12     // Clock generation
13     initial clk = 0;
14     always #5 clk = ~clk;
15
16     // Transaction object
17     fsm_transaction tr;
18
19     // Golden model states
20     state_e cs, ns;
21
22     // Stimulus and checking
23     initial begin
24         tr = new();
25
26         // Reset sequence
27         rst = 1;
28         #15 rst = 0;
29     end
```

```

30 repeat (20000) begin
31     assert(tr.randomize()); // generate random input
32     x = tr.x;
33     rst = tr.rst;
34
35     @(posedge clk);
36
37     // Sample functional coverage
38     tr.cg_x.sample();
39
40     // Compare DUT with expected
41     check_result(tr);
42 end
43
44 $display("TEST COMPLETED");|
45
46 $stop;
47 end
48
49 // Check task
50 task check_result(fsm_transaction tr);
51     golden_model(tr);
52
53     // Compare expected vs DUT
54     if (tr.y_exp != y || tr.user_count_exp != users_count) begin
55         $error("Mismatch! Expected y=%0d, count=%0d | Got y=%0d, count=%0d",
56             tr.y_exp, tr.user_count_exp, y, users_count);
57     end
58 endtask
59

```

```

60 // Golden model task
61 task golden_model(fsm_transaction tr);
62     static bit [9:0] exp_count = 0;
63
64     if (tr.rst) begin
65         cs = IDLE;
66         exp_count = 0;
67         tr.y_exp = 0;
68         tr.user_count_exp = 0;
69     end else begin
70         // next state logic
71         case (cs)
72             IDLE: ns = (tr.x==0) ? ZERO : IDLE;
73             ZERO: ns = (tr.x==1) ? ONE : ZERO;
74             ONE: ns = (tr.x==0) ? STORE : IDLE;
75             STORE: ns = (tr.x==0) ? ZERO : IDLE;
76         endcase
77
78         // Output and count update
79         if (ns == STORE) begin
80             exp_count++;
81             tr.y_exp = 1;
82         end else begin
83             tr.y_exp = 0;
84         end
85
86         tr.user_count_exp = exp_count;
87         cs = ns;
88     end
89 endtask
90
91 endmodule

```

2. Package code

```

1 package fsm_pkg;
2     // FSM states typedef
3     typedef enum logic [1:0] {
4         IDLE  = 2'b00,
5         ZERO  = 2'b01,
6         ONE   = 2'b10,
7         STORE = 2'b11
8     } state_e;
9
10    // Transaction class
11    class fsm_transaction;
12        rand bit x;
13        rand bit rst;
14        bit y_exp;
15        bit [9:0] user_count_exp;
16
17        // Constraints
18        constraint rst_c { rst dist {1 := 1, 0 := 9}; } // reset mostly inactive
19        constraint x_c   { x dist {0 := 67, 1 := 33}; } // x=0 ~67%
20
21        // ---- Functional Coverage ----
22        covergroup cg_x ; // Sampling on positive clock edge
23            coverpoint x {
24                bins zero = {0};
25                bins one  = {1};
26                bins zero_one_zero = (0 => 1 => 0); // Sequence transition bin
27            }
28        endgroup
29
30        // Constructor
31        function new();
32            cg_x = new();
33        endfunction
34    endclass
35 endpackage
36

```

3. Design code

```

1  //////////////////////////////////////
2  // Author: Kareem Waseem
3  // Course: Digital Verification using SV & UVM
4  //
5  // Description: 010-sequence-detector Design
6  //
7  //////////////////////////////////////
8  module FSM_010(clk, rst, x, y, users_count);
9      parameter IDLE = 2'b00;
10     parameter ZERO = 2'b01;
11     parameter ONE = 2'b10;
12     parameter STORE = 2'b11;
13
14     input clk, rst, x;
15     output y;
16     output reg [9:0] users_count;
17
18     reg [1:0] cs, ns;
19
20     always @(*) begin
21         case (cs)
22             IDLE:
23                 if (x)
24                     ns = IDLE;
25                 else
26                     ns = ZERO;
27             ZERO:
28                 if (x)
29                     ns = ONE;
30                 else
31                     ns = ZERO;
32             ONE:
33                 if (x)
34                     ns = IDLE;
35                 else
36                     ns = STORE;

```

```

37         STORE:
38             if (x)
39                 ns = IDLE;
40             else
41                 ns = ZERO;
42             default:    ns = IDLE;
43         endcase
44     end
45
46     always @(posedge clk or posedge rst) begin
47         if(rst) begin
48             cs <= IDLE;
49         end
50         else begin
51             cs <= ns;
52         end
53     end
54
55     always @(posedge clk or posedge rst) begin
56         if(rst) begin
57             users_count <= 0;
58         end else begin
59             if (ns == STORE) // FIXED: check ns, not cs
60                 users_count <= users_count + 1;
61         end
62     end
63
64     assign y = (ns == STORE);
65
66 endmodule

```

4. Report any bugs detected in the design and fix them
5. Snippet to your verification plan document

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FSM_1	When rst is asserted, users_count must be reset to 0 and state must go to IDLE.	Directed at the start of the simulation	-	A checker in the testbench ensures that both users_count and y are 0 after reset.
FSM_2	Detect pattern $x = 0 \rightarrow 1 \rightarrow 0$ and transition through states ZERO $\rightarrow$ ONE $\rightarrow$ STORE.	Randomization with bias towards x toggling between 0 and 1 frequently	Coverpoint to track the transition of x from 0 to 1 to 0 and states transitions (ZERO $\rightarrow$ ONE $\rightarrow$ STORE).	A checker in the testbench compares DUT outputs (y and users_count) with the golden model.
FSM_3	When in STORE state, increment users_count by 1 and set y = 1.	Randomization with forced transitions to STORE.	Coverpoint for checking when ns == STORE.	A checker ensures users_count increments correctly and y is asserted.
FSM_4	Ensure proper state transitions between IDLE, ZERO, ONE, and STORE.	Randomized x values with proper distribution.	FSM coverage on state transitions.	A checker validates that the DUT follows the FSM transition table exactly.
FSM_5	Ensure no simultaneous increment of users_count unless in STORE state.	Randomization with sequences that avoid STORE condition.	-	A checker verifies that users_count remains stable outside the STORE state.
FSM_6	Verify output y matches the expected behavior (y = 1 only in STORE).	Randomization + directed tests with specific patterns.	Coverpoint on y to confirm toggling when entering and leaving STORE.	A checker compares y with expected STORE state behavior using the golden model.

6. Do file

```

1  vlib work
2  vlog FSM_010.v  FSM_pkg.svh FSM_tb.svh  +cover -covercells
3  vsim -voptargs=+acc work.FSM_tb -cover
4  add wave *
5  coverage save FSM_tb.ucdb -onexit |
6  run -all

```

7. Code Coverage report snippets

Coverage Report by instance with details

```

=====
=== Instance: /\FSM_tb#dut
=== Design Unit: work.FSM_010
=====

```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	18	18	0	100.00%

=====Branch Details=====

Branch Coverage for instance /\FSM_tb#dut

Line	Item	Count	Source
----	----	----	-----
File FSM_010.v			
-----CASE Branch-----			
21		19816	Count coming in to CASE
22	1	5761	IDLE:
27	1	7703	ZERO:
32	1	3965	ONE:
37	1	2387	STORE:

Branch totals: 4 hits of 4 branches = 100.00%

-----IF Branch-----			
23		5761	Count coming in to IF
23	1	2410	if (x)
25	1	3351	else

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----			
28		7703	Count coming in to IF
28	1	3245	if (x)
30	1	4458	else

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
 33                               3965    Count coming in to IF
 33             1                1524
                               1524    if (x)

 35             1                2441
                               2441    else
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
 38                               2387    Count coming in to IF
 38             1                913
                               913    if (x)

 40             1                1474
                               1474    else
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
 47                               17373   Count coming in to IF
 47             1                3515        if(rst) begin
                               3515

 50             1                13858        else begin
                               13858
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
 56                               14489   Count coming in to IF
 56             1                3515        if(rst) begin
                               3515

 58             1                10974        end else begin
                               10974
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
 59                               10974   Count coming in to IF
 59             1                1842        if (ns == STORE) // FIXED: check ns, not cs
                               1842

                               9132    All False Count
```

Branch totals: 2 hits of 2 branches = 100.00%

Condition Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
Conditions	1	1	0	100.00%

====Condition Details=====

Condition Coverage for instance /\FSM_tb#dut --

File FSM_010.v

-----Focused Condition View-----

Line 59 Item 1 (ns == 3)

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(ns == 3)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(ns == 3)_0	-
Row 2:	1	(ns == 3)_1	-

Expression Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
Expressions	1	1	0	100.00%

====Expression Details=====

Expression Coverage for instance /\FSM_tb#dut --

File FSM_010.v

-----Focused Expression View-----

Line 64 Item 1 (ns == 3)

Expression totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(ns == 3)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
ROW 1:	1	(ns == 3)_0	-
ROW 2:	1	(ns == 3)_1	-

FSM Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
FSM States	4	4	0	100.00%
FSM Transitions	7	7	0	100.00%

=====FSM Details=====

FSM Coverage for instance /\FSM_tb#dut --

FSM_ID: cs

Current State Object : cs

State Value MapInfo :

Line	State Name	Value
22	IDLE	0
27	ZERO	1
32	ONE	2
37	STORE	3

Covered States :

State	Hit_count
IDLE	5802
ZERO	5844
ONE	2758
STORE	1673

Covered Transitions :

Line	Trans_ID	Hit_count	Transition
26	0	2647	IDLE -> ZERO
29	1	2758	ZERO -> ONE

code_cvr_fsm - Notepad

File Edit Format View Help

29	1	2758	ZERO -> ONE
48	2	905	ZERO -> IDLE
36	3	1673	ONE -> STORE
34	4	1085	ONE -> IDLE
41	5	1016	STORE -> ZERO
39	6	656	STORE -> IDLE

Summary	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
FSM States	4	4	0	100.00%
FSM Transitions	7	7	0	100.00%

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	16	16	0	100.00%

=====Statement Details=====

Statement Coverage for instance /\FSM_tb#dut --

Line	Item	Count	Source
----	----	----	-----
File FSM_010.v			
8			module FSM_010(clk, rst, x, y, users_count);
9			parameter IDLE = 2'b00;
10			parameter ZERO = 2'b01;
11			parameter ONE = 2'b10;
12			parameter STORE = 2'b11;
13			
14			input clk, rst, x;
15			output y;
16			output reg [9:0] users_count;
17			

17

18

reg [1:0] cs, ns;

19

20

1

19816

always @(*) begin

21

case (cs)

22

IDLE:

23

if (x)

24

1

2410

ns = IDLE;

25

else

26

1

3351

ns = ZERO;

27

ZERO:

28

if (x)

29

1

3245

ns = ONE;

30

else

31

1

4458

ns = ZERO;

32

ONE:

33

if (x)

34

1

1524

ns = IDLE;

35

else

36

1

2441

ns = STORE;

37

STORE:

38

if (x)

code_cvr_fsm - Notepad

File Edit Format View Help

```
38                                     if (x)
39                                     ns = IDLE;
40                                     else
41                                     ns = ZERO;
42                                     default: ns = IDLE;
43                                     endcase
44                                     end
45
46                                     1          17373      always @(posedge clk or posedge rst) begin
47                                     if(rst) begin
48                                     1          3515          cs <= IDLE;
49                                     end
50                                     else begin
51                                     1          13858          cs <= ns;
52                                     end
53                                     end
54
55                                     1          14489      always @(posedge clk or posedge rst) begin
56                                     if(rst) begin
57                                     1          3515          users_count <= 0;
58                                     end else begin
59                                     if (ns == STORE) // FIXED: check ns, not cs
```

```

59                                     if (ns == STORE) // FIXED: check ns, not cs
60                                     users_count <= users_count + 1;
61                                     end
62                                     end
63
64                                     1          14936      assign y = (ns == STORE);

```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Toggles	24	24	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\FSM_tb#dut --

Node	1H->0L	0L->1H	"Coverage"
clk	1	1	100.00
cs[1-0]	1	1	100.00
ns[1-0]	1	1	100.00
rst	1	1	100.00
users_count[3-0]	1	1	100.00
x	1	1	100.00
y	1	1	100.00

Total Node Count = 12
 Toggled Node Count = 12
 Untoggled Node Count = 0

Toggle Coverage = 100.00% (24 of 24 bins)

Total Coverage By Instance (filtered view): 100.00%

8. Functional Coverage report snippets

Covergroups

Covergroups							
File Bookmarks Window Help							
Covergroups							
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	
/fsm_pkg/fsm_tran...		100.00%					
TYPE cg_x		100.00%	100	100.00...			
CVP cg_x::...		100.00%	100	100.00...			
INST \fsm...		100.00%	100	100.00...			
CVP x		100.00%	100	100.00...			
bin ...		13400	1	100.00...			
bin ...		6600	1	100.00...			
bin ...		2939	1	100.00...			

cvr_fsm - Notepad

File Edit Format View Help

Covergroup Coverage:

Covergroups	1	na	na	100.00%
Coverpoints/Crosses	1	na	na	na
Covergroup Bins	3	3	0	100.00%

Covergroup	Metric	Goal	Bins	Status
TYPE /fsm_pkg/fsm_transaction/cg_x	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Coverpoint x	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Covergroup instance \fsm_pkg::fsm_transaction::cg_x	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Coverpoint x	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin zero	13400	1	-	Covered
bin one	6600	1	-	Covered
bin zero_one_zero	2939	1	-	Covered

Statement Coverage:

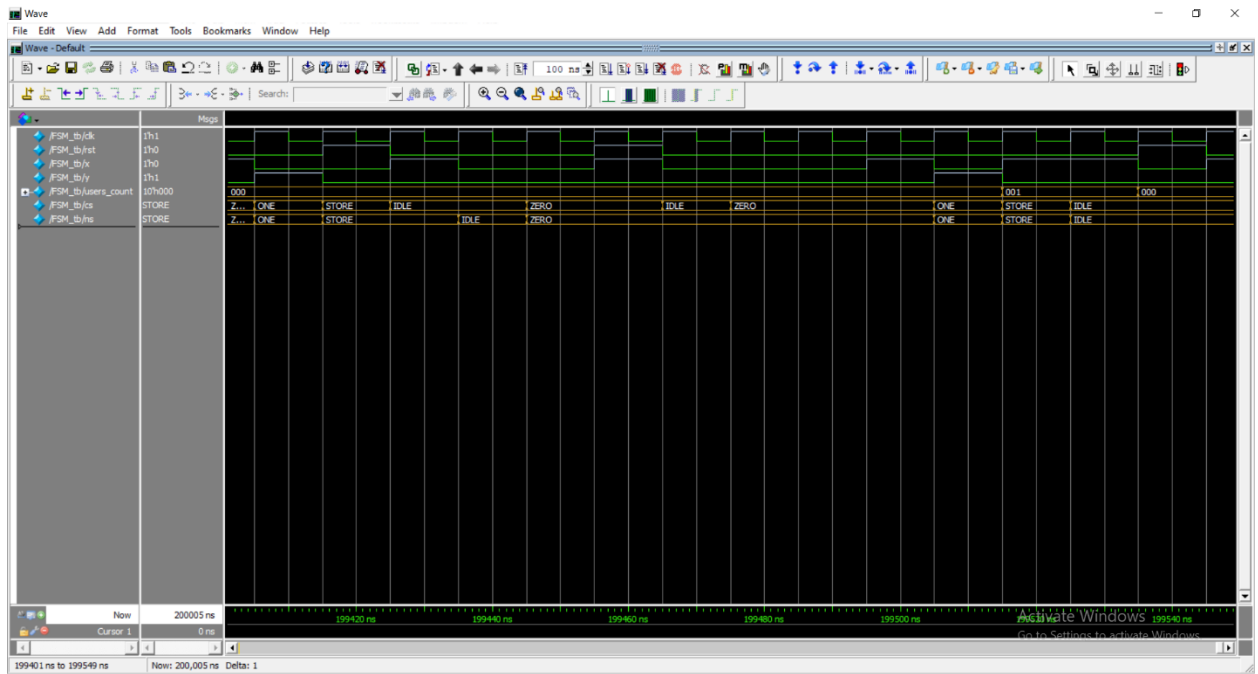
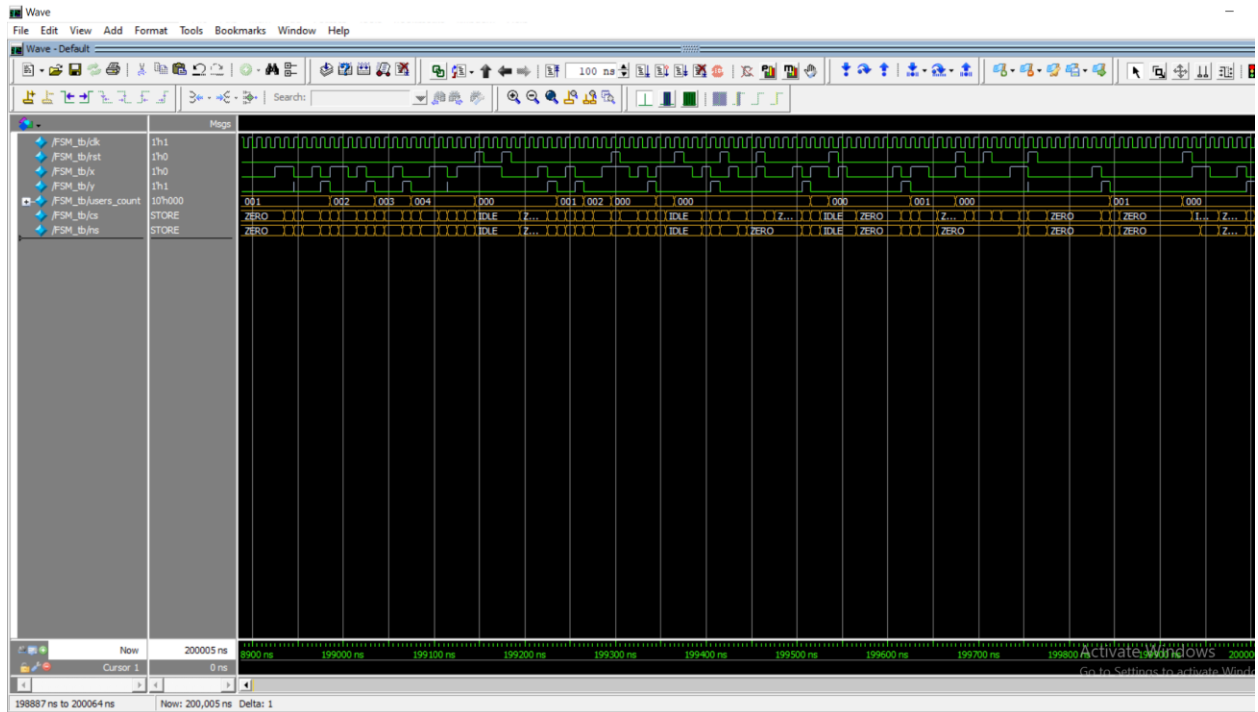
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	1	1	0	100.00%

COVERGROUP COVERAGE:

Covergroup	Metric	Goal	Bins	Status
TYPE /fsm_pkg/fsm_transaction/cg_x	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Coverpoint x	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Covergroup instance \fsm_pkg::fsm_transaction::cg_x	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Coverpoint x	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin zero	13400	1	-	Covered
bin one	6600	1	-	Covered
bin zero_one_zero	2939	1	-	Covered

TOTAL COVERGROUP COVERAGE: 100.00% COVERGROUP TYPES: 1

9. Clear and neat QuestaSim waveform snippets showing the functionality of the design



Q4

```

1  module processor_memory;
2
3      // -----
4      // Parameters
5      // -----
6      localparam int WORD_WIDTH    = 24;          // Each word is 24 bits
7      localparam int ADDRESS_SPACE = 1 << 20;     // 2^20 addresses (20-bit address)
8      localparam int RESET_ADDR    = 20'h00000;    // Reset starts at address 0
9      localparam int PROGRAM_START = 20'h00400;    // Program starts at address 0x400
10     localparam int ISR_ADDR       = 20'hFFFFFF;   // ISR at max possible address (2^20 - 1)
11
12     // -----
13     // Memory Declaration - Associative Array
14     // -----
15     logic [WORD_WIDTH-1:0] memory [int];
16     // Key: address (int), Value: 24-bit word
17
18     // -----
19     // Initialization
20     // -----
21     initial begin
22         // Fill memory with given instructions
23         memory[RESET_ADDR] = 24'hA50400; // Reset instruction
24         memory[PROGRAM_START] = 24'h123456; // Instruction 1
25         memory[PROGRAM_START + 1] = 24'h789ABC; // Instruction 2
26         memory[ISR_ADDR] = 24'h0F1E2D; // ISR instruction
27
28         // Display the total number of elements in the array
29         $display("-----");
30         $display("Memory Initialization Complete!");
31         $display("Number of elements stored in memory = %0d", memory.num());
32         $display("-----");
33
34         // Display all memory contents using foreach
35         foreach (memory[address]) begin
36             $display("Address: 0x%05h | Data: 0x%06h", address, memory[address]);
37         end
38
39         $display("-----");
40         $stop;
41     end
42
43     endmodule
44

```

```

# Compile of processor_memory.sv was successful.
VSIM(paused)> vsim -voptargs="+acc" work.processor_memory
# vsim -voptargs="+acc" work.processor_memory
# Start time: 04:36:11 on Sep 12, 2025
# ** Note: (vsim-3812) Design is being optimized...
# Loading sv_std.std
# Loading work.processor_memory(fast)
VSIM(paused)> run -all
# -----
# Memory Initialization Complete!
# Number of elements stored in memory = 4
# -----
# Address: 0x00000 | Data: 0xa50400
# Address: 0x00400 | Data: 0x123456
# Address: 0x00401 | Data: 0x789abc
# Address: 0xfffff | Data: 0x0fle2d
# -----

```

Q5

```

1  module packet_example;
2
3  // 1. Define a user-defined 7-bit type
4  typedef logic [6:0] seven_bit_t;
5
6  // 2. Define a typedef struct for the packet fields
7  typedef struct packed {
8      seven_bit_t crc;    // Bits [0:6]
9      seven_bit_t data;   // Bits [7:13]
10     seven_bit_t cmd;    // Bits [14:20]
11     seven_bit_t header; // Bits [21:27]
12 } packet;
13
14 // 3. Declare a structure variable of type packet
15 packet my_packet;
16
17 initial begin
18     // Assign the header field to 7'h5A
19     my_packet.header = 7'h5A;
20
21     // Assign example values to other fields
22     my_packet.cmd    = 7'h12;
23     my_packet.data   = 7'h34;
24     my_packet.crc    = 7'h7F;
25
26     // Display the packet contents
27     $display("Packet Contents:");
28     $display("Header = %h", my_packet.header);
29     $display("Cmd    = %h", my_packet.cmd);
30     $display("Data   = %h", my_packet.data);
31     $display("CRC    = %h", my_packet.crc);
32
33     // Display the entire 28-bit packed value
34     $display("Full Packet (28-bit) = %h", my_packet);
35 end
36 endmodule

```

```
- - - - -
# Compile of packet.sv was successful.
VSIM(paused)> vsim -voptargs="+acc" work.packet_example
# vsim -voptargs="+acc" work.packet_example
# Start time: 04:45:50 on Sep 12, 2025
# ** Note: (vsim-3812) Design is being optimized...
# Loading sv_std.std
# Loading work.packet_example(fast)
VSIM(paused)> run -all
# Packet Contents:
# Header = 5a
# Cmd    = 12
# Data   = 34
# CRC    = 7f
# Full Packet (28-bit) = fed095a
```